

Desafio Kubernetes

Da aplicação local ao Amazon EKS

Desafio Modular - Computação em Nuvem e Orquestração com Kubernetes
Relatório técnico acadêmico | Evidências de 07 de setembro de 2026

Universidade Salesiano Espírito Santo
6924106557 - Luiz Gabriel de Oliveira Ferreira
6924106421 - Yahn de Freitas Santos
6924106679 - Davi dos Santos

1. Introdução e objetivo

O trabalho demonstra o percurso de uma aplicação HTTP desde sua execução local até a disponibilização externa no Amazon EKS. O escopo inclui a construção de uma imagem Docker, sua publicação no Docker Hub, o gerenciamento de duas réplicas pelo Kubernetes, a configuração por ConfigMap e Secret e a reposição automática de um Pod excluído.

A aplicação **Materiais Hospitalares** foi mantida propositalmente simples, sem banco de dados ou regras de negócio. O objetivo é verificar a containerização, a configuração e a orquestração, com resultados sustentados pelos arquivos do projeto e pelas evidências preservadas.

2. Aplicação e teste local

A API usa Python e Flask, escuta na porta **5000** e oferece dois endpoints. A variável **AMBIENTE** identifica o contexto de execução; **APP_SECRET** é recebida em tempo de execução. A resposta da API informa apenas se o Secret está configurado, sem expor seu conteúdo. [C]

Endpoint	Resultado local registrado	Evidência
GET /	ambiente: local; aplicacao: Materiais Hospitalares; secret_configurado: true; status: online	[E1]
GET /health	status: ok	[E2]

Tabela 1 - Respostas locais observadas nas capturas originais. Fonte: [E1] e [E2], reproduzidas no Anexo A.

As imagens comprovam o acesso em **localhost:5000** e os corpos JSON. O HTTP 200 local foi informado no histórico da atividade, mas os cabeçalhos não estão visíveis nessas duas capturas. A execução local foi realizada com a imagem Docker e a porta 5000 publicada, conforme [E3].

Base documental e organização da entrega

Foram examinadas oito capturas originais, o arquivo **fase3-comandos.txt** e o código e manifestos locais. Este relatório possui **7 páginas de conteúdo**, seguidas de **4 páginas de anexos**. Cada captura aparece uma única vez; os testes externos e o self-healing são apresentados como registros textuais reais. A consolidação não executou novos testes nem operações na AWS.

3. Containerização e publicação

O **Dockerfile** utiliza a base **python:3.12-slim**, define o diretório de trabalho, instala as dependências e copia somente os arquivos necessários à aplicação. O comando de inicialização é **python app.py**. A publicação de porta no comando de execução conecta a porta 5000 do computador à mesma porta no container.

Identificação	Valor utilizado
Imagem local	desafio-kubernetes:v1
Imagem publicada	luidyon/desafio-kubernetes:v1
Registro e repositório	Docker Hub / luidyon/desafio-kubernetes
Tag	v1

Uma imagem reúne a aplicação e suas dependências. A tag identifica a versão utilizada na atividade, e o registro Docker Hub armazena a imagem para distribuição. A captura [E3] mostra **docker images**, **docker ps**, a criação da tag e o push concluído com digest. A captura [E4] confirma a tag **v1** no repositório público.

O histórico registra **docker build -t desafio-kubernetes:v1** e a execução com **AMBIENTE=local**, um Secret fictício e publicação da porta 5000. As saídas integrais de build e run não foram preservadas em capturas próprias; a imagem resultante e o container em execução estão visíveis em [E3]. Nenhum novo build ou push foi feito para produzir este relatório.

Arquivos referenciados e suas funções

Arquivo	Função no projeto
app/app.py	Implementa GET /, GET /health e leitura das variáveis de ambiente.
app/requirements.txt	Declara a dependência Flask da aplicação.
Dockerfile	Define a imagem base, instalação, cópia de código, porta e inicialização.
.dockerignore	Exclui documentação e outros arquivos do contexto de build.
k8s/deployment.yaml	Referencia a imagem publicada, define duas réplicas e injeta as configurações.
k8s/service.yaml	Expõe os Pods por Service LoadBalancer na porta 80, com destino 5000.
k8s/configmap.yaml	Declara a configuração não sensível AMBIENTE.
k8s/secret.yaml	Declara o Secret de demonstração com valor fictício.

Tabela 2 - Função dos arquivos essenciais. Fonte: código e manifestos locais [C]. O código completo será consultado no repositório GitHub informado separadamente na entrega.

4. Amazon EKS e arquitetura implantada

O cluster foi provisionado previamente e consultado durante a validação registrada em 07/09/2026. A AWS CLI autenticou o usuário **desafio-kubernetes-admin**; o kubectl utilizou o contexto desse cluster e o namespace **default**. O identificador da conta foi omitido no registro. [T, seções 1 e 8]

Componente	Estado observado
Cluster / região	desafio-kubernetes / us-east-1 / ACTIVE
Node group / capacidade	workers / 1 node t3.small
Node	ip-192-168-33-60.ec2.internal / Ready
Deployment	materiais-app / READY 2/2 / AVAILABLE 2

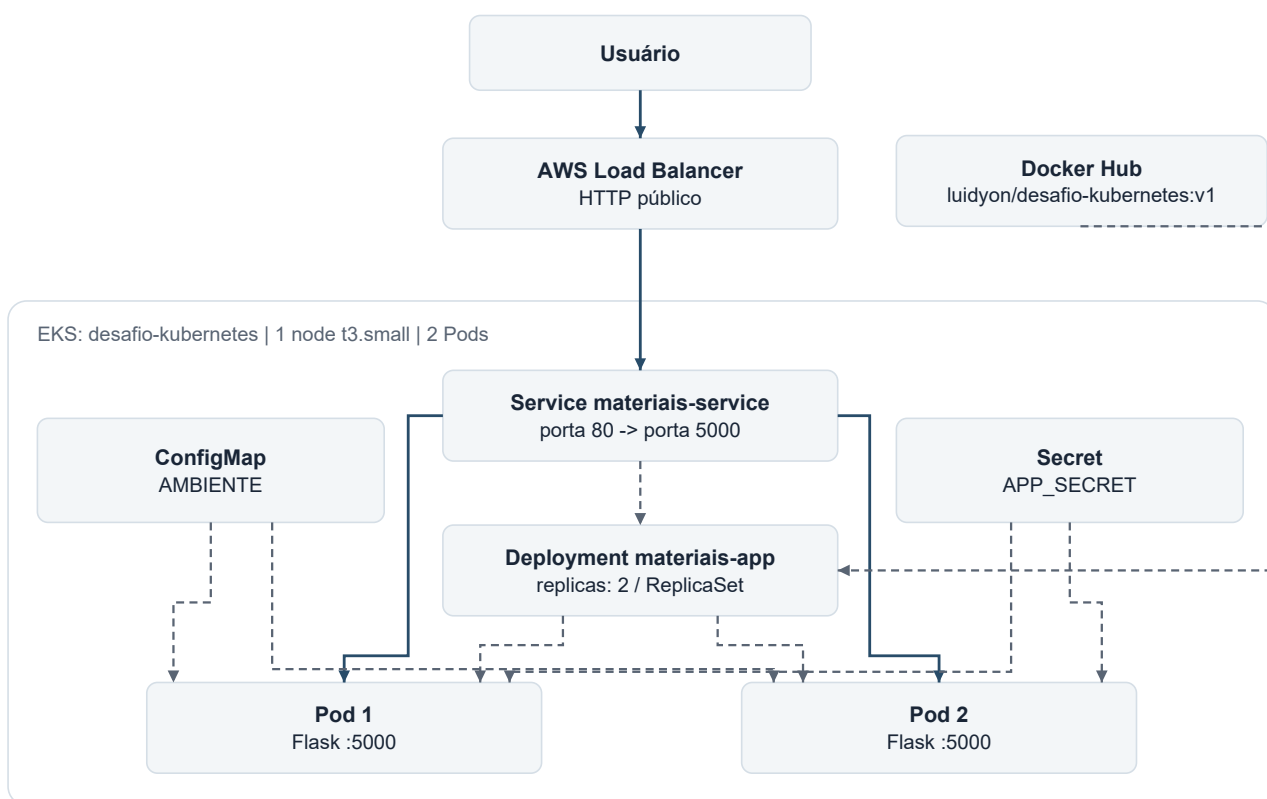


Figura 1 - Arquitetura elaborada a partir dos manifestos [C] e do estado registrado [T]. Linhas contínuas: tráfego HTTP. Linhas tracejadas: relações de configuração e gerenciamento.

O Service seleciona os Pods pelas labels; o tráfego HTTP segue do Load Balancer para o Service e os Pods. O Deployment define **replicas: 2** e gerencia o ReplicaSet, sem atuar como intermediário HTTP. Os dois Pods foram observados no mesmo node. A captura [E5] comprova o cluster ACTIVE; o node Ready está comprovado em [T]. [E6] registra o Deployment e as duas réplicas em execução.

5. Configuração e acesso externo

5.1 ConfigMap e Secret

O ConfigMap **materiais-config** e o Secret **materiais-secret** foram encontrados no namespace default. O Deployment usa **configMapKeyRef** para AMBIENTE e **secretKeyRef** para APP_SECRET. Nos dois Pods finais, inclusive no substituto, a verificação retornou **AMBIENTE=demonstracao** e **APP_SECRET=configurado**. [T, seções 2 e 8; E7]

O Secret foi listado apenas por metadados. O teste dentro dos containers verificou sua presença sem imprimir o conteúdo. A captura [E7] mostra as variáveis nos Pods; a existência dos objetos ConfigMap e Secret é documentada no registro textual.

5.2 Service LoadBalancer

Campo	Valor observado
Service / tipo	materiais-service / LoadBalancer
Selector	app=materiais-app
Porta / targetPort	80 / 5000
Endpoints após recuperação	192.168.60.154:5000 e 192.168.60.179:5000

Tabela 3 - Configuração e endpoints do Service. Fonte: [E8] e [T, seções 4 e 8].

O endereço foi obtido diretamente do Service, sem assumir um hostname fixo:

```
a22cf95f8f7a14db285c7655db7b1447-1519761716.us-east-1.elb.amazonaws.com
```

5.3 Testes HTTP externos

GET / e **GET /health** retornaram **HTTP 200 OK** antes e depois do self-healing. As quatro respostas estão preservadas em [T, seções 5, 6 e 8]. Os corpos abaixo mantêm os valores registrados, com o JSON apenas formatado para leitura.

GET / - resposta registrada

```
Status HTTP: 200 OK
{
  "ambiente": "demonstracao",
  "aplicacao": "Materiais Hospitalares",
  "secret_configurado": true,
  "status": "online"
}
```

GET /health - resposta registrada

```
Status HTTP: 200 OK
{
  "status": "ok"
}
```

6. Recuperação automática: self-healing

Antes da ação, o registro confirmou dois Pods 1/1 **Running** e sua propriedade pela cadeia **Deployment - ReplicaSet - Pods**. Às **16:58:12 de 07/09/2026 (UTC-03:00)**, foi iniciada a exclusão de somente um Pod de materiais-app, usando **kubectl delete pod** com **--wait=false**. O outro Pod e o Deployment foram preservados. [T, seção 7]

Papel	Nome completo
Pod excluído	materiais-app-86c5fbfcc9-mpvpb
Pod preservado	materiais-app-86c5fbfcc9-r4s2l
Novo Pod	materiais-app-86c5fbfcc9-prtkp

Tabela 4 - Identificação dos Pods envolvidos. Fonte: [T, seção 7].

Excertos reais do arquivo de comandos

Antes da exclusão - transcrição literal selecionada

```
Nomes antes: materiais-app-86c5fbfcc9-mpvpb, materiais-app-86c5fbfcc9-r4s2l
```

Confirmação da exclusão - saída literal

```
pod "materiais-app-86c5fbfcc9-mpvpb" deleted from default namespace
```

Resultado registrado - transcrição literal selecionada

```
Pod excluído: materiais-app-86c5fbfcc9-mpvpb
Novo Pod: materiais-app-86c5fbfcc9-prtkp
Nomes finais: materiais-app-86c5fbfcc9-prtkp, materiais-app-86c5fbfcc9-r4s2l
```

Esses trechos são apresentados como documentação textual extraída de **fase3-comandos.txt**. Não foram convertidos em capturas de terminal, recriados como screenshots ou obtidos pela repetição do teste.

Tempo observado e estado final

Observação	Resultado
Dois Pods ativos prontos	Aproximadamente 4,6 segundos, incluindo o novo Pod.
Encerramento do Pod excluído	Aproximadamente 34,2 segundos até restarem exatamente dois Pods totais.
Estado final	Deployment READY 2/2, AVAILABLE 2; dois Pods 1/1 Running.

O Pod excluído ainda apareceu brevemente como **Terminating** enquanto o substituto já estava pronto. Os tempos incluem latência de consultas e esperas de 2 segundos. Como o estado desejado é **replicas: 2**, o ReplicaSet gerenciado pelo Deployment cria um substituto ao detectar a redução da quantidade de Pods. O teste comprova a reposição de uma réplica; não mede disponibilidade contínua.

7. Discussão dos resultados

7.1 Dificuldades encontradas

O histórico da atividade relata dificuldades na configuração inicial do **Docker Desktop/WSL2**, na autenticação da **AWS CLI**, na preparação do **IAM**, no tempo de provisionamento do **EKS** e no pequeno intervalo até o **Load Balancer** responder externamente. Esses pontos são relatos do autor; não há registros detalhados de mensagens de erro, duração ou ações corretivas para atribuir causas ou soluções específicas.

7.2 Aprendizados

A execução demonstrou a relação entre imagem, tag e registro, pois a versão publicada no Docker Hub foi referenciada no Deployment. Também evidenciou a separação entre configuração não sensível e Secret, a seleção dos Pods por labels no Service e a reconciliação do número de réplicas após uma exclusão.

O self-healing permitiu distinguir dois momentos: a prontidão do novo Pod e o encerramento completo do Pod anterior. Os resultados também mostram que duas réplicas no mesmo node não equivalem à tolerância à perda desse node. As consultas e respostas HTTP comprovam o estado observado, sem substituir testes de carga ou de disponibilidade contínua.

7.3 Custos e cuidados

Os valores efetivos de custo não foram apurados. O ambiente foi mantido ativo apenas durante os testes e a coleta das evidências. Após a conclusão, o Service LoadBalancer foi removido e o cluster EKS foi excluído com `eksctl delete cluster`.

Recursos AWS removidos e conferidos. A verificação final registrou No clusters found; não foram encontradas instâncias EC2 ativas vinculadas ao cluster nem Classic ELB ou ELBv2 remanescentes em us-east-1. A Access Key e o usuário IAM temporários do laboratório também foram removidos.

7.4 Limitações

A API é didática, sem persistência de dados ou regras de negócio. O cluster observado possuía um único node. Não foram registrados ensaios de carga, falha do node ou monitoramento contínuo durante a exclusão do Pod. As capturas locais não exibem os cabeçalhos HTTP 200, e não há saídas próprias de build e run; essas lacunas estão discriminadas na conferência de critérios.

8. Conclusão

O exercício demonstrou o fluxo da aplicação local até o acesso externo no Amazon EKS, com duas réplicas, variáveis recebidas nos Pods e reposição automática após a exclusão de uma réplica. Os dois endpoints responderam HTTP 200 antes e depois do teste. Após a coleta das evidências, os recursos AWS foram removidos e a ausência de cluster EKS, instâncias EC2 e Load Balancers remanescentes foi verificada.

9. Conferência dos critérios e fontes

A conferência abaixo utiliza os critérios informados para esta entrega. Não há, nos arquivos do projeto, uma cópia separada do enunciado integral com pesos de avaliação. Ausência de screenshot não foi tratada como ausência de resultado quando há saída de comando real disponível.

Critério	Fonte e comprovação	Situação
Aplicação e teste local	[C], [E1] e [E2]: endpoints e corpos JSON. HTTP 200 local consta apenas do histórico.	Com ressalva
Docker: imagem e container	[E3]: imagem, container em execução e porta publicada. Sem saídas próprias de build/run.	Com ressalva
Docker Hub	[E3] e [E4]: push concluído e tag v1 publicada.	Comprovado
Amazon EKS e node	[E5]: ACTIVE; [T], seções 1 e 8: node Ready, grupo e tipo.	Comprovado
Deployment: duas réplicas	[E6] e [T]: READY 2/2, AVAILABLE 2 e dois Pods Running.	Comprovado
ConfigMap e Secret	[T], seção 2: existência; [E7] e [T], seção 8: variáveis nos dois Pods.	Comprovado
Service LoadBalancer	[E8] e [T]: tipo, hostname, porta 80 e destino 5000.	Comprovado
Acesso externo e /health	[T], seções 5, 6 e 8: HTTP 200 e JSON nos dois endpoints.	Comprovado
Recuperação automática	[T], seção 7: antes, exclusão única, novo Pod, tempos e estado final.	Comprovado
Arquitetura	Figura 1, com base nos manifestos e recursos observados.	Documentado
Dificuldades e aprendizados	Relatos do histórico e análise dos resultados, seção 7.	Documentado
Custos e cuidados	Recursos mantidos apenas durante os testes; valores efetivos não apurados.	Parcial
Remoção dos recursos	Service e cluster removidos; conferência final sem EKS, EC2 ou ELB remanescentes.	Comprovado
Conclusão	Seção 8: resultados e limites sustentados pelas fontes.	Documentado

Referências internas

[C] Código da aplicação, Dockerfile e quatro manifestos em k8s/. Arquivos essenciais incluídos no backup; repositório GitHub informado separadamente na entrega.

[T] docs/evidencias/fase3-comandos.txt. Coleta de 07/09/2026, entre 16:54 e 17:00 (UTC-03:00). Registro preservado integralmente no backup; excertos no corpo do relatório.

[E1]-[E8] Capturas originais em docs/evidencias/, numeradas de 01 a 08. Os anexos identificam cada arquivo, descrevem o que está visível e reproduzem as imagens sem edição de conteúdo.

Anexo A - Validação local e Docker Hub

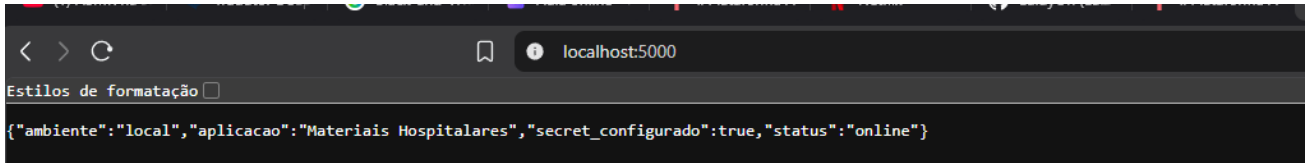


Figura A1 [E1] - GET / em localhost:5000: ambiente local, aplicação online e indicação de Secret configurado. Fonte: 01-aplicacao-local.png. A captura mostra o corpo da resposta, sem os cabeçalhos HTTP.

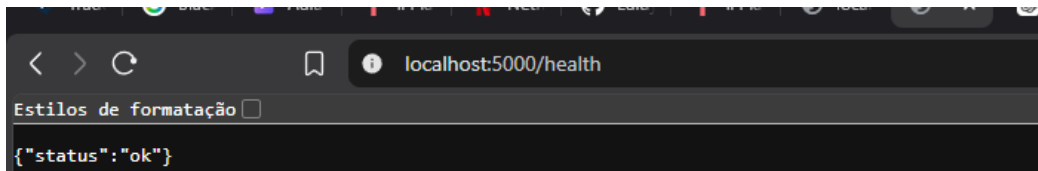


Figura A2 [E2] - GET /health local retornando status: ok. Fonte: 02-health-check.png.

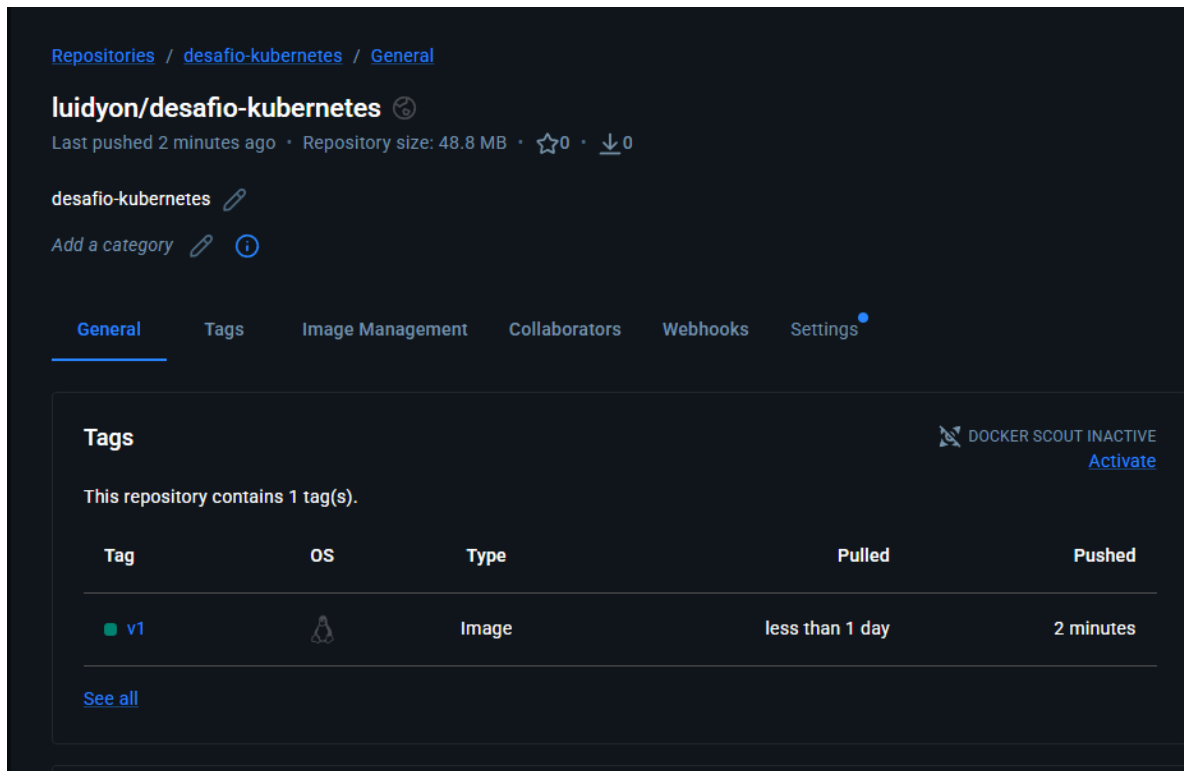


Figura A3 [E4] - Repositório luidyon/desafio-kubernetes no Docker Hub com a tag v1. Fonte: 04-docker-hub-v1.png.

Anexo B - Docker e cluster EKS

```
PROBLEMAS 1 SAÍDA CONSOLE DE DEPURACÃO TERMINAL PORTAS

PS D:\BRbri\Documents\Desafio_Kubernetes>
PS D:\BRbri\Documents\Desafio_Kubernetes> docker images
>> docker ps

IMAGE                ID                DISK USAGE  CONTENT SIZE  EXTRA
desafio-kubernetes:v1 12c96b9ec1a3      208MB       51.2MB        U
CONTAINER ID         IMAGE             COMMAND      CREATED        STATUS        PORTS
NAMES
f32064a7bfb5        desafio-kubernetes:v1 "python app.py" 7 minutes ago  Up 7 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
stupefied_chandrasekhar
PS D:\BRbri\Documents\Desafio_Kubernetes> docker tag desafio-kubernetes:v1 luidyon/desafio-kubernetes:v1
PS D:\BRbri\Documents\Desafio_Kubernetes> docker images

IMAGE                ID                DISK USAGE  CONTENT SIZE  EXTRA
desafio-kubernetes:v1 12c96b9ec1a3      208MB       51.2MB        U
luidyon/desafio-kubernetes:v1 12c96b9ec1a3      208MB       51.2MB        U
PS D:\BRbri\Documents\Desafio_Kubernetes> docker push luidyon/desafio-kubernetes:v1
The push refers to repository [docker.io/luidyon/desafio-kubernetes]
4998433e7ed8: Pushed
44136fa355b3: Pushed
dbd0b7849e6c: Pushed
6310eb16bf42: Pushed
1560cfed01dc: Pushed
56235e424563: Pushed
4dfdfac63636: Pushed
f9a5baa71cdd: Pushed
1e826474610b: Pushed
b1d07d5a819c: Pushed
v1: digest: sha256:12c96b9ec1a3de106b522f5ed4f85f0cd7ba9d02394ecd4d7a6c1fa285fd3fff size: 856
PS D:\BRbri\Documents\Desafio_Kubernetes>
```

Figura B1 [E3] - Terminal com docker images, docker ps, criação da tag luidyon/desafio-kubernetes:v1 e push concluído. Fonte: 03-docker-images-ps-push.png. A mesma captura reúne as evidências sem duplicação.

DescribeCluster		
Nome	Status	Versao
desafio-kubernetes	ACTIVE	1.34

Figura B2 [E5] - Consulta DescribeCluster com desafio-kubernetes ACTIVE. Fonte: 05-eks-node-ready.png. Apesar do nome do arquivo, a imagem não exibe o node Ready; essa comprovação está em [T, seções 1 e 8].

Anexo C - Réplicas e configuração nos Pods

```
PS D:\BRbri\Documents\Desafio_Kubernetes> kubectl get deployments
>> kubectl get pods -o wide
NAME                READY    UP-TO-DATE    AVAILABLE    AGE
materiais-app        2/2      2             2            65m
NAME                READY    STATUS    RESTARTS   AGE    IP                NODE                                NOMINATED NODE    READINESS GATES
materiais-app-86c5fbbcc9-prtkp  1/1      Running    0          31m    192.168.60.154    ip-192-168-33-60.ec2.internal      <none>            <none>
materiais-app-86c5fbbcc9-r4s2l  1/1      Running    0          65m    192.168.60.179    ip-192-168-33-60.ec2.internal      <none>            <none>
```

Figura C1 [E6] - Deployment `materiais-app` com `READY 2/2` e `AVAILABLE 2`, seguido dos Pods `prtkp` e `r4s2l` em `1/1 Running`. Fonte: 06-deployment-dois-pods.png. A imagem preserva os nomes completos e os IPs.

```
PS D:\BRbri\Documents\Desafio_Kubernetes> $pods = kubectl get pods -l app=materiais-app -o jsonpath='{.items[*].metadata.name}'
>>
>> foreach ($pod in $pods.Split(" ")) {
>>     Write-Host "`nPOD: $pod"
>>     kubectl exec $pod -- printenv AMBIENTE
>>     kubectl exec $pod -- sh -c 'if [ -n "$APP_SECRET" ]; then echo "APP_SECRET=configurado"; else echo "APP_SECRET=ausente"; fi'
>> }

POD: materiais-app-86c5fbbcc9-prtkp
demonstracao
APP_SECRET=configurado

POD: materiais-app-86c5fbbcc9-r4s2l
demonstracao
APP_SECRET=configurado
PS D:\BRbri\Documents\Desafio_Kubernetes>
```

Figura C2 [E7] - Validação de `AMBIENTE=demonstracao` e da presença de `APP_SECRET` em ambos os Pods, sem imprimir o segredo. Fonte: 07-configmap-secret-pods.png. A existência dos objetos `ConfigMap` e `Secret` é comprovada separadamente em [T, seção 2].

Anexo D - Exposição da aplicação

```
>> kubectl describe service materiais-service
NAME:                 materiais-service
TYPE:                 LoadBalancer
CLUSTER-IP:          10.100.140.24
EXTERNAL-IP:         a22cf95f8f7a14db285c7655db7b1447-1519761716.us-east-1.elb.amazonaws.com
PORT(S):              80:31179/TCP
AGE:                  64m
SELECTOR:             app=materiais-app
IPFAMILY:             IPv4
IPFAMILIES:           IPv4
IP:                   10.100.140.24
IPS:                  10.100.140.24
LOADBALANCER INGRESS: a22cf95f8f7a14db285c7655db7b1447-1519761716.us-east-1.elb.amazonaws.com
PORT:                 http 80/TCP
TARGETPORT:           5000/TCP
NODEPORT:             http 31179/TCP
ENDPOINTS:            192.168.60.179:5000,192.168.60.154:5000
SESSION AFFINITY:     None
EXTERNAL TRAFFIC POLICY: Cluster
INTERNAL TRAFFIC POLICY: Cluster
EVENTS:               <none>
PS D:\BRbri\Documents\Desafio_Kubernetes>
```

Figura D1 [E8] - Service materiais-service do tipo LoadBalancer, hostname externo, porta 80/TCP, targetPort 5000/TCP e endpoints 192.168.60.179:5000 e 192.168.60.154:5000. Fonte: 08-loadbalancer.png.

Os testes HTTP e a recuperação automática estão documentados nas seções 5 e 6 do relatório por meio dos registros reais de fase3-comandos.txt. Não foram acrescentadas capturas artificiais para esses resultados.